Programming the Pico Computer 3 in C

Fuzix on the RP2350B

What this is

Everything a C programmer needs to reach the facilities this port adds to Fuzix: the graphics modes, the off-screen framebuffer, text at pixel positions, sound — both the BBC synthesiser and streamed PCM — the shared maths library, the joystick and ADC, and the 8 MiB of PSRAM.

None of it is a library call. The kernel owns the hardware and exposes it through **ioctls on `/dev/sys`**, so everything below is the same three lines:

```
#include <fcntl.h>
#include <sys/ioctl.h>
#include "pico_ioctl.h"          /* the numbers and structures */

int sys = open("/dev/sys", O_RDWR);
ioctl(sys, GFXIOC_MODE, &mode);
```

`pico_ioctl.h` is the authority. If this document and that header disagree, the header is right.

Two ways to write C here

There are two, and they are genuinely different environments.

On the board: `cc`

```
$ cc -o prog.bc prog.c
$ ./prog.bc
$ cc -r prog.c          build it and, if it built, run it
```

The Fuzix C compiler runs on the machine itself and produces bytecode with native ARM Thumb spans, executed by `bcrun`. This is what `mmbc` targets and what most PC3 programs are.

What you get:

- headers: `stdio.h`, `stdlib.h`, `string.h`, `assert.h`, `limits.h`, `stddef.h`,

`math.h`, `ctype.h`, `stdint.h`, `time.h` — in `/usr/lib/cc/include`, describing what `bcrun` provides rather than what Fuzix's own libc does.

- a C library subset resolved **by name at load time**: `printf`, `sprintf`, `fprintf`, `fopen`/`fclose`/`fread`/`fwrite`/`fgetc`/ `fgets`/`fputc`/`fputs`/`fseek`/`ftell`/`feof`, `malloc`/`calloc`/ `realloc`/`free`, the `str*` and `mem*` family, `atoi`/`atof`/`atol`/ `strtol`/`strtod`, `open`/`close`/`read`/`write`/`lseek`/`creat`/ `remove`/`rename`/`unlink`, `exit`, `rand`/`srand`, `time`. From v0.10 the `str*` and `mem*` family are **newlib's**, taken from the ARM toolchain's own `libc.a` — tuned assembly rather than the byte-at-a-time loops Fuzix ships generically. A `memcpy` that used to run at about nine cycles a byte now moves a word at a time. Nothing in a program needs changing to get it, but a program whose profile was dominated by string work will be noticeably quicker.
- two of our own: **`adval(n)`** and **`time_us()` / `time_us64()`**.
- the MMBasic runtime, callable from plain C: `mmb_runtime.h` declares the `mm_*` entry points `bcrun` resolves by name — PRINT and formatting, the core strings, files, and the graphics crossings — and the drawing primitives are static functions in one header per primitive: `mmb_gfx_circle.h`, `mmb_gfx_box.h`, `mmb_gfx_rbox.h`, `mmb_gfx_triangle.h`, `mmb_gfx_arc.h`, `mmb_gfx_text.h`, `mmb_gfx_map.h` (batch helpers in `mmb_gfx_pts.h`; `mmb_gfx.h` is the umbrella that includes the lot; `mmb_gpio.h` is SETPIN/PIN). Since v0.19 the pure-computation families follow the same pattern rather than living inside `bcrun`: `mmb_sort.h` (SORT), `mmb_array.h` (whole-array operations, REDIM's arithmetic, the `MATH()` reductions), `mmb_lstring.h` (LONGSTRING), `mmb_datetime.h` (DATE/*TIME*/EPOCH), `mmb_data.h` (DATA/READ) and `mmb_misc.h` (GOSUB, BIT/FLAG, BIN2STR, *TRIM*/FIELD$, the MAP() arithmetic) — include the one you need and you get the same code a translated BASIC program gets. Include only what you use: the compiler drops an unused static, but the rule counts names rather than reachability, so a recursive primitive survives inside any header that carries it — the include is the granularity.

Because names resolve at load time, **declaring a function is all the header you need**:

```
extern long time_us(void);        /* no header required */
```

The catch: **`ioctl` is not in that table**. A `bcrun` program cannot call it, so it cannot reach the graphics or PSRAM ioctls directly. What it gets instead is `adval()` and `time_us()`, which `bcrun` implements on its behalf. To drive the display from a `bcrun` program, either write MMBasic and let `mmbc` translate it, or call the `mm_*` runtime and the `mmb_gfx_*.h` primitives from C directly (the bullet above) — either way the runtime does the ioctls.

Cross-compiled: native ARM binaries

Everything in `Kernel/platform/platform-rpipico/utils/` is built this way, and this is the path for a program that needs the full syscall interface:

```
arm-none-eabi-gcc -mcpu=cortex-m33 -Os -fno-builtin \
    -isystem $FUZIX/Library/include -c prog.c
arm-none-eabi-ld $FUZIX/Library/libs/crt0_armm0.o prog.o -o prog \
    -L$FUZIX/Library/libs -lcarmm0 -pie -static -no-dynamic-linker \
    -z max-page-size=4 -T $FUZIX/Library/elfexe32.ld
```

or just add it to `utils/Makefile` and `make`. These get Fuzix's own libc, `ioctl`, and every syscall. **Everything in the rest of this document assumes this path.**

Ask for more stack with `-z stack-size=N` if you recurse; the default is 8 KB and the ELF loader honours the request up to 64 KB.

Floating point is soft float, and that is not negotiable for most programs. The RP2350's FPU is granted, but no FP register state is saved across a context switch, so exactly one process on the machine may execute FP instructions. That process is the audio player, and it holds the sound device for as long as it runs — the device lock *is* the FPU lock. Do not add `-mfpu` to anything else. For maths that has to be fast, use the shared library below: it is double precision, it runs on the DCP, and it costs no lock.

`printf` can print one, as of v0.13. It could not before: the `%e`, `%f` and `%g` conversions sat behind a build option no target defined, and the function they called was declared and never written, so `printf("%f", x)` put the letter `f` on the screen. `%j` and `%z` were missing in the same way. Both directions work now — `scanf` reads a float too.

A bytecode program's `printf` is a different implementation (bcrun's own, since the library had nothing to offer it) and gained `%e` and `%g` at the same time. Before that they printed themselves *and* did not consume their argument, so every conversion after one of them read the wrong slot — `printf("%g %d", 1.5, 42)` printed `%g` and then a piece of the double.

Worked examples on disc

Read these before writing anything; they are all short.

program	shows
`utils/gfxtest.c`	modes, palette, `GFXIOC_BLIT`
`utils/linebench.c`	`GFXIOC_PIXEL` against `GFXIOC_PIXELS`, with timings
`utils/ripple.c`	per-pixel drawing versus a shadow buffer

program	shows
`utils/bmtest.c`	`GFXIOC_BITMAP` and `GFXIOC_RECT`, verified by readback
`utils/saveimage.c`, `loadimage.c`	reading and writing the screen
`utils/loadjpg.c`	picojpeg, a block at a time - memory that does not grow with the picture
`utils/loadpng.c`, `upng_pc3.c`	upng, which must inflate the whole image, so it uses a PSRAM arena; and its `-s` mode, which writes a sprite to stdout for another process to read
`utils/memprobe.c`	how much memory a process can actually have
`utils/allocbench.c`	the cost of a PSRAM arena allocation
`utils/pcmtest.c`	the PCM sound sink, from a generated tone
`utils/playmp3.c`	a complete decoder: file → PCM → the ring
`utils/libmbench.c`	the shared maths library against the local one

The memory a program gets

- The process pool is **340 KB**, and one process may use nearly all of it — `memprobe` reports about **292 KB**. The ceiling is not a constant: a grow must leave room for the largest *other* process to be resident (`largest_neighbour()` in `swapper.c`), so it moves with what else is running.
- Up to **64 processes**.
- Memory is packed in 4 KB chunks at actual size. A swapped-out process gets a PSRAM allocation of exactly its own size.
- `sbrk()` grows the break and fails when there is no room. Probing with `sbrk(4096)` in a loop until it refuses is a legitimate way to find your ceiling — `bbcbasic` does exactly that.
- **There is no MMU.** A wild pointer corrupts the kernel and takes the machine down with no diagnostic. This is the single most important thing to know.

Graphics

Modes

```
int mode = 7;
ioctl(sys, GFXIOC_MODE, &mode);      /* pass a POINTER to an int */
```

mode	resolution	colours	raster
0, 3	640×256, 1bpp	2	1024×768
1, 4	320×256, 4bpp	16	1024×768
2, 5	160×256, 4bpp	16	1024×768
7	320×240, 4bpp	16	640×480
0xFF	back to the 80×40 text console		640×480

Modes 0–5 are the BBC Micro set. **Mode 7 is not teletext** — it is 320×240 in 16 colours, and it is what MMBasic calls `MODE 2`.

Switching *within* a raster keeps the monitor locked; crossing between 640×480 and 1024×768 restarts the scanout and the monitor resyncs.

Geometry, so nothing has to hardcode it:

```
struct gfx_info gi;
ioctl(sys, GFXIOC_INFO, &gi);
/* gi.width, gi.height, gi.stride, gi.bpp, gi.mode */
```

Colour

Everything speaks **RGB888** and the kernel converts to the mode's own colours — you never deal in colour indices.

```
ioctl(sys, GFXIOC_COLOUR, (void *)0xFF8000L);   /* the VALUE, not a pointer */
```

Mode 7's sixteen colours are MMBasic's RGB121 set: red and blue are 0 or 255, green has four levels. Anything else maps to the nearest, which is worth knowing before you ask for `RGB(r, g, 99)` and get black — a blue of 99 is nearer 0 than 255 every time.

Drawing

```
ioctl(sys, GFXIOC_PIXEL, (void *)GFX_PIXEL_PACK(x, y));   /* current colour */
int c = ioctl(sys, GFXIOC_GETPIXEL, (void *)GFX_PIXEL_PACK(x, y));

struct gfx_rect r = { x1, y1, x2, y2 };
ioctl(sys, GFXIOC_RECT, &r);        /* filled, current colour */
```

`GFX_PIXEL_PACK` puts the coordinates in the ioctl *argument*, so there is nothing to copy in and nothing to validate: measured at 1.30 µs against 1.488 µs for an ordinary ioctl. Its y is 9 bits, so it stops at 511 — use the batch calls for the 768-line modes.

A syscall costs 1.3 µs and a pixel store costs 15 ns. Anything with more than a handful of points should be batched:

```
struct gfx_pt pts[512];
struct gfx_batch b = { .count = n, .flags = 0,
                       .items = pts, .colours = NULL };
ioctl(sys, GFXIOC_PIXELS, &b);      /* a run of points */
ioctl(sys, GFXIOC_RECTS, &b);       /* a run of rectangles: gfx_rc[] */
```

`colours` may be `NULL` for "all in the current colour", or one RGB888 per item. Up to `GFX_BATCH_MAX` (1024) items. A 312-point line costs 433 µs one at a time and 71 µs batched.

The arrays are read where they lie — nothing is copied into the kernel.

Bitmaps and text

```
struct gfx_bitmap gb = { x, y, w, h, scale, 0, fg, bg, bits };
ioctl(sys, GFXIOC_BITMAP, &gb);
```

Bits are MSB-first, row-major — **exactly MMBasic's font packing**, so fonts interchange. `bg = -1` is transparent. Source up to `GFX_BITMAP_MAX`.

For text there is no need to carry a font — there are nine of them in the kernel, MMBasic's own, and they are `const` so they live in flash and cost a program nothing:

```
struct gfx_text gt = { x, y, scale, font, fg, bg, len, str };
int endx = ioctl(sys, GFXIOC_TEXT, &gt);
```

This draws a run of characters and returns the x it ended at. `bg = -1` leaves the paper alone. Up to `GFX_TEXT_MAX` (256) characters, one crossing. `font` is 1..9 as MMBasic numbers them, and **0 means 1**, so code written before there were fonts to choose from still gets the console's:

font	cell	characters
1	8×12	224 — the console's own
2	12×20	95
3	16×24	95
4	10×16	224
5	24×32	95
6	32×50	11 — the digits only
7	6×8	96
8	4×6	96
9	8×10	95

A character the font does not have fills its cell with the paper colour, which is what MMBasic does. That is not a corner case: with font 6 it is every character that is not a digit.

Ask for the cell rather than assuming it — laying text out against the wrong metrics is the whole failure mode here:

```
struct gfx_fontinfo fi = { .font = 3 };
ioctl(sys, GFXIOC_FONTINFO, &fi);
/* fi.width, fi.height, fi.first, fi.count, fi.nfonts */
```

`fi.width` comes back 0 for a font that does not exist.

A font of your own

`GFXIOC_FONTDEF` is the mirror of `GFXIOC_FONTADDR`: that one tells you where a kernel font is, this one tells the kernel where yours is.

```
static const unsigned char myfont[] = {
    8, 8, '0', 3,               /* width, height, first char, count */
    /* then count glyphs, width*height bits each, MSB first */
};
struct gfx_fontdef fd = { .font = 10, .addr = (uint32_t)(uintptr_t)myfont,
                          .bytes = sizeof myfont };
ioctl(sys, GFXIOC_FONTDEF, &fd);
```

It is then font 10 to `GFXIOC_TEXT`, `FONTINFO` and `FONTADDR`, and the renderer treats it exactly like a built-in one — any cell size, because every metric comes out of those four header bytes.

Nothing is copied: the glyphs stay in your image and the kernel reads them where they lie, which works here for the same reason `FONTADDR` does — no MMU, so an address is an address. Numbers **10 to 16**; 1 to 9 are the built-in nine and are refused, because the console and every other program draw with them.

The registration belongs to your process and goes when it exits or execs, and it is invisible to everyone else — ask for font 10 in another program and you get its own or nothing. That is not tidiness: every process here loads at the same address, so one program's font pointer is a plausible address inside the next one, and a slot left behind would draw glyphs out of whatever is running now.

The call refuses a `bytes` that disagrees with the header, an address that is not yours, and a width × height that is not a multiple of 8 — the condition that makes the glyph bits plain MSB-first bytes.

Reading the framebuffer

Writing pixels has seven entry points, two of them batched. Reading had one — `GFXIOC_GETPIXEL`, a pixel at a time, RGB888, about 2.5 µs each. `GFXIOC_BLITRD` is the other half:

```
struct gfx_blit gb = { .offset = y * stride, .len = stride, .buf = row };
ioctl(sys, GFXIOC_BLITRD, &gb);
```

Same struct as `GFXIOC_BLIT`, same bounds check, and the same target — `display_fb_target()`, so a program drawing into the layer reads the layer back. **Native bytes**, exactly as `GFXIOC_BLIT` writes them: 4bpp high nibble is the left pixel, 1bpp MSB is the left pixel. Take the stride and depth from `GFXIOC_INFO`.

These are MMBasic's two readers under other names — `ReadBufferFast` here, `ReadBuffer` for `GFXIOC_GETPIXEL`, which converts through the palette. Use `GETPIXEL` to ask what colour something is; use this to *look at* a lot of pixels. A 320-pixel row is one call and 160 bytes rather than 320 system calls and 800 µs.

Raw bytes are palette indices, and the palette and its nearest-match are the kernel's, so `GFXIOC_COLOUR` returns the index it mapped your RGB888 to:

```
int idx = ioctl(sys, GFXIOC_COLOUR, (void *)0xFF0000);  /* red -> index */
```

It sets the drawing colour as a side effect, which costs nothing since every drawing call pushes its own colour first.

Reading the glyphs yourself

If you are driving a display of your own — an SPI panel the kernel knows nothing about — you want the glyph bits, not a syscall per character. Ask where they are:

```
struct gfx_fontaddr fa = { .font = 3 };
ioctl(sys, GFXIOC_FONTADDR, &fa);
const unsigned char *fp = (const unsigned char *)fa.addr;
```

The same bargain as `PICOIOC_LIBM`: there is no MMU, and the fonts are `const` so they are in XIP flash. `fa.addr` is an address you can read directly, it will not move, and nothing needs to be mapped or freed. `fa.bytes` is the whole extent, header and glyphs, so you can bound your reads; both come back 0 for a font that does not exist.

The layout at `fa.addr` is MMBasic's:

```
int w = fp[0], h = fp[1], first = fp[2], count = fp[3];
const unsigned char *g = fp + 4 + (c - first) * w * h / 8;
/* bit y * w + x of g is the pixel at (x, y), MSB first,
   packed continuously - no padding between rows or glyphs */
int on = (g[(y * w + x) / 8] >> (7 - ((y * w + x) & 7))) & 1;
```

`w * h` is a multiple of 8 for all nine fonts, which is what makes that plain MSB-first byte packing rather than a bit stream.

This is what BASIC's `MM.INFO(FONT ADDRESS n)` is built on.

The palette

A 4bpp mode stores a colour *number* per pixel and the palette says what each number is, so remapping recolours everything already drawn without touching a byte of the framebuffer. This is MMBasic's `MAP`.

```
ioctl(sys, GFXIOC_MAP, (void *)((index << 24) | rgb888));   /* collect */
ioctl(sys, GFXIOC_MAPCTL, (void *)GFX_MAP_SET);             /* apply   */
ioctl(sys, GFXIOC_MAPCTL, (void *)GFX_MAP_RESET);           /* default */
```

`GFXIOC_MAP` changes nothing on screen: entries are collected and `GFX_MAP_SET` moves the whole palette across **during the vertical blanking**, so a new palette arrives between frames rather than recolouring the picture in instalments. 16-colour modes only.

The colour stored is RGB332 — the byte the scanout puts on the wire — reduced by truncation, as MMBasic's `RGB332()` does.

Note that this does not change which entry a colour lands on. In the 320x240 mode that is fixed bit extraction (MMBasic's `RGB121`: one bit of red, two of green, one of blue), deliberately independent of the palette, so a program can remap freely and go on naming colours the same way.

```
ioctl(sys, GFXIOC_SCROLL, (void *)((rows << 24) | rgb888));
```

Scrolls the write target: `rows` signed in the top byte, positive up. This is the same call the console uses for its own scrolling, so a program and the shell move the picture the same way.

The off-screen framebuffer

Draw off-screen, show it in one go. This is MMBasic's `FRAMEBUFFER`.

```
ioctl(sys, GFXIOC_FBOPEN, (void *)1);    /* claim it */
ioctl(sys, GFXIOC_FBSEL,  (void *)1);    /* draw off-screen */
   ... every drawing call above now goes to the buffer ...
ioctl(sys, GFXIOC_VSYNC,  (void *)0);    /* optional: top of frame */
ioctl(sys, GFXIOC_FBCOPY, (void *)0);    /* buffer -> screen */
ioctl(sys, GFXIOC_FBSEL,  (void *)0);    /* back to the screen */
ioctl(sys, GFXIOC_FBOPEN, (void *)0);    /* give it back */
```

Things to know:

- **The layer is owned.** `FBOPEN` fails with `EBUSY` if another process holds it, `EINVAL` on a board with no PSRAM. It is released automatically on exit, on exec, and on a mode change — so a program that dies does not leave the machine drawing into nowhere.
- **The write target is per process.** Anything else that draws while you are between frames — another program, the shell — still goes to the screen. You cannot have your picture scribbled on.

- **`CREATE` belongs after `MODE`.** A mode change discards the buffer, because its contents are in the geometry of the mode being left.
- `FBCOPY` takes 1 to copy screen→buffer instead.

What it costs, measured on the board in mode 7:

full-screen fill, to the screen (SRAM)	1.49 ms
the same into the buffer (PSRAM)	3.63 ms
copy the 38,400-byte buffer to the screen	**0.73 ms** (53 MB/s)

So a redraw-and-show frame is about 4.4 ms against the 17 ms the display gives you. `GFXIOC_VSYNC` before the copy holds a loop to the refresh rate — 59 fps with room for roughly three times as much drawing.

`GFXIOC_VSYNC` holds the CPU while it waits, and nothing else on the machine runs. The kernel does not preempt inside a system call, so a wait of up to a frame stops every other process — audible at once if anything is playing. Where that matters, wait in slices instead:

```
while (!ioctl(sys, GFXIOC_VSYNCTRY, (void *)2000))  /* microseconds */
        ;
```

`GFXIOC_VSYNCTRY` spins for at most the budget you give it (capped at 20 ms) and returns 1 if the top of blanking arrived, 0 if it did not — so the loop is back in user mode between tries, where the tick can take the CPU away and give it to somebody who needs it. `FRAMEBUFFER MERGE` in the BASIC runtime does exactly this, which is why compositing no longer starves the audio.

Sound

Four channels, BBC `SOUND` semantics.

```
struct snd_cmd s = { .chan = 1, .amp = -15, .pitch = 100, .dur = 20 };
ioctl(sys, SNDIOC_SOUND, &s);    /* -1/EAGAIN if that queue is full */
ioctl(sys, SNDIOC_ENV, envbytes); /* 14 bytes: N,T,PI1..3,PN1..3,AA,AD,AS,AR,ALA,ALD */
ioctl(sys, SNDIOC_QUIET, 0);      /* silence everything */
```

`amp` is -15..0 for volume, or 1..16 to use `ENVELOPE n`. The channel word carries the `&1x` flush and `&Sxx` sync bits.

Playing your own samples

The same I2S engine will take decoded PCM instead of the synthesiser. Open the stream, write 16-bit frames as fast as the ring will take them, and the driver's DMA does the rest — nothing has to be refilled from a main loop, which is what lets music carry on while another program runs.

```
struct snd_pcm cfg = { .rate = 44100, .channels = 2, .bits = 16 };
struct snd_buf sb;
struct snd_stat st;

if (ioctl(sys, SNDIOC_PCMOPEN, &cfg) < 0)   /* EBUSY: someone else has it */
        ...
sb.base = samples; sb.len = nbytes;
n = ioctl(sys, SNDIOC_PCMWRITE, &sb);       /* returns bytes ACCEPTED */
ioctl(sys, SNDIOC_PCMSTAT, &st);            /* space, queued, underruns */
ioctl(sys, SNDIOC_PCMCLOSE, 0);
```

- A **short write is normal**, not an error: it means the ring is full and you should come back. The ring holds 256 KB, about 1.5 seconds at 44100 stereo — deep on purpose, because a Fuzix process can be swapped out for tens of milliseconds and the audio must not notice.
- **Do not poll with `usleep` — use `SNDIOC_PCMWAIT`.**

  ```
  ioctl(sys, SNDIOC_PCMWAIT, (void *)low_water);  /* bytes, not a ptr */
  ```

  It sleeps until the ring has drained to `low_water` and returns 0, or -1 if you do not hold the stream; a signal wakes it too, which is what a stop request needs. This exists because `usleep` **cannot sleep for less than 100 ms** — it rounds up to the timer wheel's decisecond — so a player that polls has to keep more than 100 ms of audio queued to cover its own sleep, and everything queued is latency before the next sound is heard. The kernel knows when the ring drains, so it does the waiting, on the 5 ms tick.

  It also **lets the woken player run**: a timeslice here is `MAXTICKS`, which is half a second, so a compute-bound program would otherwise hold the processor while the ready player's queue emptied. That, not buffering, is what made the MOD player stutter under load — and it is why a queue of 93 ms is now enough where 186 ms was not.
- **Mono is expanded by the driver**, so a mono source costs you nothing and halves the traffic into the ring.
- `underruns` counts half-buffers the interrupt had to fill with silence. It is the objective test of whether your buffering is deep enough, and much better than "it sounded all right".
- To play the tail out, poll `PCMSTAT` until `queued` is zero and close then; `PCMCLOSE` itself is immediate and drops whatever is left.

One process at a time. There is one I2S engine, so the stream belongs to whoever opened it: a second `PCMOPEN` gets `EBUSY`, and writes and closes from anyone else are refused. This is deliberate — two players sharing the ring interleave their samples, and it sounds precisely as bad as that suggests.

```
int who = ioctl(sys, SNDIOC_PCMOWNER, 0);   /* the pid playing, or 0 */
```

`SNDIOC_PCMOWNER` is how a program that did not start the player can find it: send it `SIGINT` to stop the music tidily. A player that dies without closing strands nothing — the kernel hands the stream back when its owner is gone, so `SIGKILL` is safe too.

`SOUND` and PCM are mutually exclusive, as they are in MMBasic: opening the stream silences the synthesiser and closing it hands the state machine back.

The shared maths library

`sin`, `cos`, `exp` and the rest live in kernel flash and are called **directly** — there is no MMU, so kernel flash is in the same address space as your program. Ask for the table once:

```
void *tbl;
ioctl(sys, PICOIOC_LIBM, &tbl);       /* check magic and version first */
```

It is double precision and uses the RP2350's DCP, which makes it about **2.7× faster** than the soft-float library linked into a program, and it costs nothing in program memory. `libm_table.c` documents the layout and the errno contract (these do not report domain errors).

Check the magic and version before calling anything. An old binary on a new kernel must fail rather than call the wrong slot — that is what the version is for.

Pins, the joystick and the ADC

These are **not** syscalls. Claim what you want through the pin lock, then read and write the registers yourself — there is no MMU on this board, so a pin costs a store rather than the 1.488 µs an ioctl costs. `<sys/pc3io.h>` has the registers and the claim wrappers:

```
#include <sys/pc3io.h>

pc3_claim(PLK_PIN, 4);          /* one syscall, once */
pc3_pin_out(4);
pc3_pin_high(4);                /* one store */

pc3_claim(PLK_PIN, 34);
pc3_pin_in(34, 1);              /* pulled up */
if (!pc3_pin_get(34)) ...       /* the joystick's bit 0, active low */

pc3_claim(PLK_ADC, 0);
pc3_claim(PLK_PIN, PC3_ADC_GPIO(1));
pc3_adc_enable();
pc3_adc_pin(1);
v = pc3_adc_read(1) << 4;       /* ADVAL's 16-bit convention */
```

Claimable: header pins GP0–GP7, GP26, GP34–GP46; I2C1; SPI0; the twelve PWM slices; the ADC. Everything else belongs to the board. A claim you already hold succeeds; someone else's gives `EBUSY`; anything not on that list gives `EINVAL`. **Exiting releases everything and resets the pins**, so a program that dies driving a relay does not leave it driven — which is the reason the kernel is involved at all.

Two traps, both of which produce code that looks right and does nothing:

- **`PADS_BANK0.ISO` resets to 1.** Until it is cleared the pad is disconnected — no drive, no read, and a pull-up does nothing. Every setup function in the header clears it last. If you write your own, do the same.
- **Never read-modify-write SIO's `GPIO_OUT` or `GPIO_OE`.** They are shared with the kernel and with core1's display. Use the SET/CLR/XOR registers, and remember GP32–GP47 are a second bank whose registers interleave with the first.

`utils/locktest.c` exercises the whole thing, including killing a process that holds a pin and checking the pin comes back.

Counting inputs, and measuring a pulse

These are the one pin family that is **not** yours to poll. GP4 to GP7 are the pins the kernel's own GPIO interrupt sits on, and the counting and the timestamps happen inside it — so a program may be descheduled for half a second, which is this machine's timeslice, and still read back an exact answer. Any other pin gives `EINVAL`.

The ioctls are on `/dev/gpio` and carry a `struct cntreq`:

```
#include <sys/pc3io.h>

int g = open("/dev/gpio", O_RDWR);
struct cntreq c;

c.pin = 5;
c.arg = 1000;                           /* gate, in ms */
ioctl(g, GPIOC_CNT_FIN, &c);            /* frequency on GP5 */

ioctl(g, GPIOC_CNT_READ, &c);           /* c.val is the answer */
ioctl(g, GPIOC_CNT_OFF, &c);
```

ioctl	`arg`	what `GPIOC_CNT_READ` then answers
`GPIOC_CNT_FIN`	gate in ms, 1–100000	Hz over the last **completed** gate — 0 until the first one finishes
`GPIOC_CNT_CIN`	1 rising, 2 falling, 3 both; 4 and 5 are 1 and 2 with the other pull	the live 64-bit edge count
`GPIOC_CNT_PER`	cycles to average, 1–10000	the period in ms

`GPIOC_CNT_SET` stores `c.val` into the counter and is **`CIN` only**; anything else gives `EINVAL`. `GPIOC_CNT_OFF` stops the counting and drops the pull. One mode per pin, and the kernel undoes all of it when the process ends however it ends.

A short gate answers sooner and in coarser steps: at `arg = 100` an `FIN` reading moves in 10 Hz steps.

One pulse: edge capture

`Pulsin(` and `Distance(` in BASIC are this underneath. `GPIOC_CNT_CAP` with `capreq.arg = 1` arms a pin and `0` disarms it; `GPIOC_CNT_CAPRD` reads back a ring of `PC3_CAP_RING` (16) timestamps in microseconds, with `seq` counting edges since arming and bit k of `lvl` giving the level after `us[k]`.

Arming does **not** touch the pin's configuration — whatever set the pin up is what gets measured — and only one pin may be armed at a time.

Driving WS2812 and arbitrary timed output

The kernel loads five fixed programs into PIO1 at boot, beside the I2S program that carries the sound, and reserves one state machine and one DMA channel for programs to use. Claim them, fill a buffer with timing words, point the DMA at it and let it run: a long stream costs the machine nothing while it plays.

```
pc3_claim(PLK_PIO, PIOOUT_PLK_IDX);      /* the reserved state machine */
pc3_claim(PLK_DMA, PIOOUT_DMA_CH);       /* and its DMA channel */
pc3_claim(PLK_PIN, 7);                   /* the output pin */
```

The program to run is chosen by its origin in PIO1's instruction memory: `PIOOUT_ORG_BS` (bitstream, driven), `PIOOUT_ORG_BSOC` (bitstream, open-collector), and `PIOOUT_ORG_WSO`, `PIOOUT_ORG_WSB`, `PIOOUT_ORG_WSS` for the three WS2812 timings. Set the state machine's divider from `PIOOUT_CLKDIV`, which is what puts `BITSTREAM`'s durations into the reference's own units.

The DMA must never read process memory. The swapper rearranges the process pool in 4 KB chunks on every context switch, so a local array's bytes move whenever its owner sleeps or is preempted, and the DMA would then read whichever process's chunks had landed underneath. The kernel reserves a buffer in PSRAM, which never moves. Ask where it is, copy the words in — a store, because there is no MMU — and point the DMA there:

```
struct pioout_buf b;
ioctl(g, GPIOC_PIOOUT_BUF, &b);          /* b.addr, b.words */
```

`PIOOUT_BUF_WORDS` (10000) is the ceiling. The `PLK_PIO` claim is what arbitrates the buffer between programs.

Two hard rules, both of which produce code that looks right:

- **PIO1's `CTRL` register is shared with the I2S state machine.** Touch it only through the atomic set and clear aliases (`PC3_PIO_SET`, `PC3_PIO_CLR`); a plain read-modify-write there can stop the audio. Only the atomic spellings appear in the header.
- **Pins GP0–GP7 and GP26 only.** PIO1's `GPIOBASE` is pinned to 0 by the I2S pins, so GP34–GP46 are outside its window. The kernel asserts this at boot.

`pc3_pioout_stop()` in the header shuts the machine down; `utils/piooutttest.c` and `utils/striptest.c` exercise both programs.

ADVAL, and the clock

```
int v = ioctl(sys, PICOIOC_ADVAL, (void *)n);
```

n	reads
0, 1–4	**gone** — joystick and ADC are pin work, see above
-5..-8	free slots in sound queue 0–3
-9	microsecond counter, 31 bits in the return value
-10	the full 64-bit counter: pass an 8-byte buffer whose low word holds -10

BASIC's `ADVAL(0)` and `ADVAL(1)`–`ADVAL(4)` are unchanged: BBC BASIC reads the pins itself now, and returns exactly what it used to.

PSRAM

8 MiB, outside the process image, reached as a raw address.

```
struct psram_req rq = { .len = 64 * 1024 };
ioctl(sys, PSRAMIOC_ALLOC, &rq);      /* rq.base is the address */
ioctl(sys, PSRAMIOC_REALLOC, &rq);    /* rq.base in, the NEW base out */
ioctl(sys, PSRAMIOC_FREE, (void *)rq.base);

struct psram_stat st;
ioctl(sys, PSRAMIOC_STAT, &st);       /* total, free, largest */
```

- Allocation is 4 KB granular and **released on exec and exit**; a fork leaves the arena with the parent.
- **`REALLOC` may move the block** — newlib copies when it cannot extend in place — so rebuild any interior pointers.
- The address is raw and unprotected; there is no MMU.
- An alloc+free pair costs about **5.1 µs**, against 350 ns for an in-process `malloc`. Ask once and sub-allocate, rather than per call.
- **PSRAM is roughly 2.4× slower than SRAM** for scattered access, but a bulk sequential copy runs at 53 MB/s. Put big, streamed things there; keep hot, randomly-accessed things in your own memory.

Networking

Unlike everything else in this manual, **most of this is not an ioctl**. Sockets are Fuzix's own, so a C program uses the calls you already know:

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *)&addr, sizeof(addr));
write(fd, req, n);
while ((n = read(fd, buf, sizeof buf)) > 0)
        ...
close(fd);
```

`AF_INET` only. `SOCK_STREAM`, `SOCK_DGRAM` and `SOCK_RAW` all work — `ping` is a raw socket. Servers work: `bind`, `listen`, `accept`.

What is missing, and what to do instead

There is no `select()` and no `poll()`. They are not compiled into this kernel, so you cannot wait on several descriptors at once.

Non-blocking sockets do work, which covers most of what you would have wanted `select` for:

```
fcntl(fd, F_SETFL, O_NDELAY);
n = read(fd, buf, sizeof buf);
if (n < 0 && errno == EAGAIN)
        /* nothing there yet - go and do something else */
```

Any socket operation that would have slept returns -1 with `EAGAIN` instead: `read`, `write`, `connect` and `accept` alike. So a program can round-robin over several sockets itself, at the cost of a spin — put a `sleep` or some other work in the loop rather than burning the processor, because there is nothing to block on. That advice is measured, not manners: on a TLS socket a hot `O_NDELAY` read loop starves the kernel of the time it needs to *decrypt* what has arrived — six million polls in thirty seconds returned nothing on a socket a blocking read drained at once. Sleep between polls and the data appears.

There is no resolver in the C library. `gethostbyname` is not in the libc the on-board `cc` links; the network tools that resolve names (`dig`, `htget`, `ntpdate`) carry their own. A C program takes a dotted quad, reads `/etc/resolv.conf` and asks the nameserver itself (the DNS query is ~40 lines — `dig`'s source in `Applications/netd` is the worked example), or runs `dig` and reads its output.

Forking is the other answer and often the better one: a Fuzix process is cheap, so a server can fork a child per connection, and a program that must watch a socket *and* the keyboard can fork one process for each and talk over a pipe.

There is no `setsockopt`. `SO_REUSEADDR` is therefore set on every TCP socket by the kernel, because without it a server could not rebind its own port for two minutes after it exited.

`accept()` blocks and returns a new descriptor, as usual. Closing that descriptor closes only that connection; the listener stays up.

Bringing the radio up

This part *is* ioctls on `/dev/sys`, because it is the radio rather than the socket layer. `pico_ioctl.h` is the authority.

```
#include "pico_ioctl.h"

struct net_join j;
strcpy(j.ssid, "MYNETWORK");
strcpy(j.key,  "MYPASSWORD");
j.auth = 2;                                 /* WPA2-AES */
ioctl(sys, NETIOC_UP, &j);                  /* blocks ~0.5s, once */

struct net_status st;
ioctl(sys, NETIOC_STATUS, &st);             /* poll until st.ready */

ioctl(sys, NETIOC_DOWN, 0);
```

`NETIOC_UP` uploads 230 KB of firmware to the chip and does not return until that is done — about half a second during which nothing else runs, including the keyboard. Association afterwards is asynchronous: poll `NETIOC_STATUS` rather than expecting `NETIOC_UP` to mean "connected". `st.present` is 0 on a

Pico Computer 2, where `NETIOC_UP` returns `ENODEV` without powering anything, because those pins are that board's SD chip select and LED.

Ordinary programs do not need any of this — `wifi -f` has done it before your program runs.

TLS

A TLS socket is an ordinary socket with two differences:

```
#define IPPROTO_TLS 254
#define SIOCTLSHOST 0x0420

int fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TLS);
ioctl(fd, SIOCTLSHOST, "example.com");   /* BEFORE connect */
connect(fd, ...);
```

The name must be set before `connect`, and it is used twice: as the SNI extension in the handshake, and as the name the certificate has to match. `connect()` carries an address and nothing else, which is why this is an ioctl.

After that it is `read`, `write` and `close` on a descriptor. That is deliberate: the same code serves plain TCP and TLS, which is what will let `mmbc` emit one implementation for both.

Certificates are not checked unless a bundle has been loaded — see `tlsca` in the user manual. A program can load one itself:

```
struct net_ca ca;
ca.buf = pem;                           /* PEM, NUL-terminated */
ca.len = pemlen + 1;                    /* the NUL COUNTS */
int checked = ioctl(sys, NETIOC_TLSCA, &ca);
```

The length **must include the terminator**: that is how mbedtls tells PEM from DER, and without it the bundle is read as DER and rejected whole. `len` of 0 clears the bundle. The call returns 1 if certificates are checked from then on and 0 if they are not.

A failed certificate surfaces as `ECONNREFUSED` from `connect`, which is honest about the outcome and says nothing about the cause. If a connection fails only when a bundle is loaded, that is why.

Loopback

`127.0.0.1` works, and so does the machine's own address, so a program can talk to a server on the same board.

Miscellaneous

```
ioctl(sys, PICOIOC_FLASH, 0);            /* reboot into BOOTSEL */
ioctl(sys, PICOIOC_KBDMAP, "uk");        /* US UK DE FR ES BE */

int board;
ioctl(sys, PICOIOC_BOARD, &board);       /* 2 or 3 */
```

`PICOIOC_BOARD` answers which machine this is — the same number the boot banner prints, from the same detection (the DS3231's 32 kHz on GP27). It exists because there was no other way to ask: the kernel knew, and only the banner ever said. A program that wants to name itself needs it — MMBasic's `MM.DEVICE$` is the first caller, and reports `Fuzix on PC2` or `Fuzix on PC3`.

An older kernel does not have it, so check the return value and pick a default rather than trusting whatever was in the variable.

Traps worth knowing

No MMU. Said once already, worth saying twice. A bad pointer in a user program takes the whole machine down with no message.

A panic blanks the HDMI screen. `plt_monitor()` resets core1 and with it the scanout, so a panic looks like a dead display. **The message only ever goes out of the serial port.** If a program kills the machine, read the UART.

Text and graphics share the framebuffer. In a graphics mode the console draws into the screen buffer, so `printf` from your program lands on the picture and scrolls it when the line wraps. Use `GFXIOC_TEXT` for text you want positioned, and keep `printf` for the serial console.

To have the screen to yourself, turn the console's display half off:

```
ioctl(sysfd, PICOIOC_CONMIRROR, (void *)0);   /* 1 puts it back */
```

The uart half keeps working, so `printf` still reaches a terminal — this is what MMBasic's `OPTION CONSOLE SERIAL` does. **The claim belongs to the process that made it.** The kernel gives the display back when *that* process ends, and a child running meanwhile — a decoder like `loadimage`, or anything else you `fork` and `exec` — leaves the screen alone. It did not always: any exec used to hand the mirror back on the owner's behalf, which painted the console's cursor into the owner's picture as an 8×12 cell and quietly re-enabled the console over the top of it.

Timing is contended. core1 DMAs scanlines out of SRAM continuously, so RAM bandwidth is shared. This cuts both ways: it is why the kernel now executes from flash at no measurable cost, and why a benchmark that touches the framebuffer will not be reproducible to the last percent.

The compiler is small. `cc` on the board has real limits — deeply nested expressions and very large functions can exceed it. It also has a threshold beyond which a function stops being compiled to native ARM and falls back to bytecode, which is worth 2–3× on a hot loop. If a small change makes a program suddenly slower, that is the first thing to suspect; `BCRUN_BYTECODE=1` forces bytecode so you can compare.

The preprocessor is the board's own. Cross-compiled builds use `gcc -E`, so anything that works there is not evidence that it works here. C89 is what to expect: v0.10 fixed `?:` inside `#if`, `#line`, `__LINE__` inside `#if`, and argument expansion before `##` pastes — none of which had ever been tested, because no host build reaches this code. Measured C conformance on the board is 160 of 175 tests.

Unbounded recursion takes the machine down. `bcrun` checks the stack on entry to a bytecode function and stops the program cleanly, but a function the compiler turned into native ARM has a three- instruction prologue with no check in it — and a small recursive function is exactly what gets translated. A missing terminating condition is a reset, not an error message.

Where things live

```
Kernel/platform/platform-rpipico/
    pico_ioctl.h          every ioctl, and the authority
    display.c/.h          modes, primitives, the framebuffer
    console.c             the terminal, and the font
    sound.c               the synthesiser and the PCM sink
    libm_table.c          the shared maths library and its ABI
    swapper.c             the process pool
    psram.c, arena.c      PSRAM and the arena allocator
    utils/                the worked examples above
    PC3-GFX-DESIGN.md     why the graphics interface is shaped as it is
    PC3-FLASH-PLAN.md     what runs from flash and what may not
    FS32-FORMAT.md        the on-disk filesystem format, and its authority
    BUILDING-PC3.md       building the kernel and the card
```